

E-commerce System MVP Design

Product Manager Assignment

Candidate

May 11, 2026

Contents

1	Product Design	2
1.1	User Roles and Core Scenarios	2
1.2	MVP Feature List and Priority	2
1.3	Core Process Design	3
1.3.1	Process 1: Customer Order and Payment Flow	3
1.3.2	Process 2: Order Shipment Process	4
1.4	Simplified PRD	4
2	Code / Technical Understanding	6
2.1	API Understanding Questions	6
2.2	Simple Logic Reading Question	7
2.3	Order Status Understanding	8
2.4	Bug / Exception Analysis	8
3	Data and Metrics	9
3.1	Core Metrics Design	9

1 Product Design

1.1 User Roles and Core Scenarios

Role	Core Scenarios
Customer / Buyer	Browse product lists, view product details, add items to cart, modify cart quantities, submit orders, select payment method and complete payment, view personal order list and details, track shipping status, confirm receipt.
Operations Staff	Manage products (create, online/offline, edit information). Manage SKUs and inventory (adjust stock). Process orders (view order list, mark orders as shipped, update tracking number). Handle simple after-sales (approve/reject refund requests).
Platform Admin	Manage operations staff accounts and permissions. View core business data (e.g., GMV, order volume, conversion rates). Handle abnormal orders that operations cannot resolve (e.g., force cancel, force refund).

Table 1: User Roles and Core Scenarios

1.2 MVP Feature List and Priority

Module	Feature	Priority	Rationale
Customer Side	User Registration/Login	P0	Identify users, associate orders and cart data.
Customer Side	Product List / Detail Page	P0	Core entry for browsing and obtaining product info.
Customer Side	Shopping Cart	P1	Improves bulk purchase experience; P1 but important for conversion. P0 can use "Buy Now".
Customer Side	Submit Order	P0	Start of core transaction flow.
Customer Side	Integrated Payment (simulated/real)	P0	Completes transaction loop. Initial version can use sandbox or simulated success/failure.
Customer Side	Order List / Details	P0	Allows users to view order status, foundation for after-sales.
Customer Side	Cancel Order	P1	Improves user experience, reduces invalid orders.
Customer Side	Product Search / Category Filter	P2	Not essential when few products, can iterate later.
Admin Side	Product Management (CRUD)	P0	Products must exist to sell. Core: create, edit, online/offline.
Admin Side	SKU / Inventory Management	P0	Manage quantities for different specifications, prevent overselling.
Admin Side	Order Management (list/details)	P0	View orders, handle shipping and after-sales. Update order status to "Shipped", fulfill transaction.

Admin Side	Shipment Management	P1	In MVP, operations can update directly in order management.
Admin Side	User Management	P2	Can be done via DB or simple page in MVP.
Admin Side	Data Dashboard	P2	Can be added later.

Table 2: MVP Feature List and Priorities

1.3 Core Process Design

1.3.1 Process 1: Customer Order and Payment Flow

A. Normal Flow:

1. User clicks "Buy Now" on cart or product detail page.
2. System redirects to "Confirm Order" page, showing product info, quantity, amount, shipping address.
3. User selects/enters shipping address, selects payment method (e.g., simulated payment), clicks "Submit Order".
4. Backend creates order with status `PENDING_PAYMENT` and locks inventory (pre-deduction).
5. Order created successfully, redirect to "Payment" page.
6. User confirms amount, clicks "Pay Now", calls payment interface.
7. Payment succeeds, payment gateway returns success callback.
8. System updates order status to `PAID`, actually deducts inventory.
9. Redirect to "Payment Success" page, user can view order details.

B. Exception Scenarios:

1. **Insufficient inventory:** When user submits order, real-time inventory is less than purchase quantity. API rejects, page shows "Insufficient stock, please adjust quantity", order not created, no inventory lock.
2. **Payment failure:** Payment gateway returns failure (e.g., insufficient balance, network issue). API returns failure message. Page shows "Payment failed: [reason]". Order stays `PENDING_PAYMENT`. User can retry payment from order details.
3. **Payment timeout:** Order created more than 30 minutes (example) without payment. Scheduled task scans orders with `PENDING_PAYMENT` older than 30 minutes, updates status to `CANCELLED`, releases locked inventory. User sees "Order cancelled" when trying to pay.
4. **Product taken offline but still in cart:** Operation staff takes product offline. When user enters cart, backend returns product info with status. Frontend grays out offline product, shows "Product offline", checkbox disabled, cannot checkout. When submitting order, backend re-checks all products' status for `ON_SALE`; if any offline, rejects order with message "Some products are invalid, please re-order".

C. Status Changes:

[No status] (cart) → `PENDING_PAYMENT` (order submitted) → `PAID` (payment success) / `CANCELLED` (payment timeout or user cancel) / `PENDING_PAYMENT` (payment failure)

1.3.2 Process 2: Order Shipment Process

A. Normal Flow:

1. Operations staff in admin panel, filter orders with status PAID.
2. Click "Ship" button next to order, enter shipment page.
3. Page shows order products and shipping address.
4. Staff enters carrier name and tracking number.
5. Click "Confirm Shipment".
6. Backend updates order status to SHIPPED and saves tracking info.
7. Frontend shows success, order list status updates.
8. System sends shipment notification SMS/push to customer.

B. Exception Scenarios:

1. **Invalid tracking number format:** System can validate with regex for common carriers; if mismatch, show "Please enter a valid tracking number", prevent shipment.
2. Duplicate shipment: Order status already SHIPPED or COMPLETED, "Ship" button not shown or backend rejects.
3. **Network timeout:** After clicking "Confirm Shipment" and timeout, prevent duplicate submission. Frontend button disabled after click, show success or failure based on final API response.

C. Status Changes: PAID → SHIPPED (operations ship)

1.4 Simplified PRD

Feature: Admin Product Management

1. Feature Background The current system has no product entry capability, so the transaction foundation cannot be built. Operations staff need a backend tool to create, edit, and manage all product information, including different SKUs and inventory, to ensure the frontend store has products to sell with accurate information.

2. User Goals As an operations staff member, I want to:

1. Quickly create a new product, including basic info, images, and multiple specifications.
2. Modify product price, inventory, and online/offline status at any time.
3. Conveniently view all products in a list.
4. Goal: Ensure product information is accurate and responsive to market changes.

3. Page Fields Basic Product Information

Field	Type	Rules/Limits	Required
Product Name	Text input	Max 100 characters	Yes
Product Description	Rich text editor	Supports text/images, max 5000 characters	No
Main Product Image	Image upload	Supports jpg/png, single ≤5MB, at least 1 image	Yes
Product Detail Images	Multi-image upload	Same as above	No
Category	Dropdown	Select from preset category tree	Yes

Table 3: Basic Product Information Fields

SKU List

Field	Type	Rules/Limits
SKU Specification (e.g., Color/Size)	Specification name+value	Supports multiple groups, Cartesian prod
SKU ID	Auto-generated	Not editable, unique identifier
Selling Price (cents)	Number input	Integer ≥0
Market Price (cents)	Number input	Integer ≤ selling price
Inventory Quantity	Number input	Non-negative integer
SKU Image	Image upload	Different images for different specification

Table 4: SKU Fields

Status Control

Field	Type	Rules/Limits	Required
Online/Offline Toggle	Toggle button	Controls whether product is visible and sellable on frontend	Yes

Table 5: Status Control Fields

4. Business Rules

1. A product must have at least one SKU.
2. When product status is "Online", it is visible on frontend. When inventory of all SKUs is zero, frontend shows "Out of stock".
3. When product is taken offline, it remains in existing user carts but is grayed out and unselectable.
4. Deleting a product is logical deletion (soft delete), not shown on frontend but visible in admin.

5. Error Messages / Exception Handling

1. Save failed: Network or backend validation failure, show "Save failed, please try again".
2. Missing required field: Show red error message below field, e.g., "Please enter product name".
3. Image upload failed: Show "Image upload failed, please check format/size or try again".
4. Inventory reduction below locked quantity: When operations try to set inventory lower than total locked by unpaid orders, show "Operation failed, new inventory cannot be lower than reserved quantity: XX".

6. Acceptance Criteria

1. Operations can successfully create a product with basic info, main image, and one SKU, then bring it online. The product is searchable and visible on frontend.
2. Operations can successfully create a product with both "Color" and "Size" specifications; the system auto-generates 4 SKUs, each with editable price and inventory.
3. Operations can take an online product offline; the frontend product detail page shows "Product offline" and cannot be purchased.
4. After operations modify a SKU's price, the frontend product detail page shows the updated price for that SKU immediately.
5. After operations set a SKU's inventory to 0, that SKU becomes "Out of stock" on frontend and cannot be selected or purchased.

2 Code / Technical Understanding

2.1 API Understanding Questions

Given API response for product detail page:

```
{
  "code": 0,
  "message": "success",
  "data": {
    "productId": 10001,
    "productName": "Bluetooth Earphones",
    "status": "ON_SALE",
    "price": 19900,
    "originalPrice": 29900,
    "stock": 25,
    "salesCount": 132,
    "images": [...]
  }
}
```

1. **Displayable information:** Product ID, name, status (e.g., "On Sale"), selling price (199.00 yuan), original price (299.00 yuan), inventory (25), sales count (132), image list.
2. **Price display:** Show as 199.00 yuan or ¥199.00. Backend returns in cents to avoid floating-point precision issues.
3. **If status = OFF_SALE:** Main product info displayed normally. "Buy" or "Add to Cart" buttons disabled or grayed out. Show prominent message: "Product offline". "Buy Now" button does not redirect to order confirmation.
4. **If total stock = 0:** Main "Buy" or "Buy Now" button disabled. Button text changed to "Out of stock" or "Temporarily out of stock". Show message on detail page: "Insufficient stock, cannot purchase at this time".
5. **When selecting "Black" SKU:** Frontend should make a new API request, or the initial API response should include all SKU data. When SKU selected, displayed price, inventory, and possibly SKU image should switch to corresponding data for the black SKU.

6. Missing fields:

- **skuList**: Needed to display different specifications (color, size) with their prices, inventory, and SKU images. Without this, page cannot implement multi-spec selection.
- **category**: Knowing product category helps with breadcrumb navigation, recommendations, SEO.
- **ratings** or **commentCount**: Reviews are key for purchase decisions; missing means no reputation display.
- **shippingInfo**: Users need to know shipping cost before ordering; missing makes order confirmation logic incomplete.
- **description**: Detailed product description area is core for conversion; completely missing from API.

2.2 Simple Logic Reading Question

Given pseudo-code:

```
function canSubmitOrder(productStatus, skuStock, buyCount){
  if (productStatus !== 'ON_SALE'){
    return { canSubmit: false, reason: 'Product is off sale' };
  }
  if (skuStock <= 0){
    return { canSubmit: false, reason: 'Insufficient inventory' };
  }
  if (buyCount > skuStock){
    return { canSubmit: false, reason: 'Purchase quantity exceeds available inventory' };
  }
  return { canSubmit: true, reason: "" };
}
```

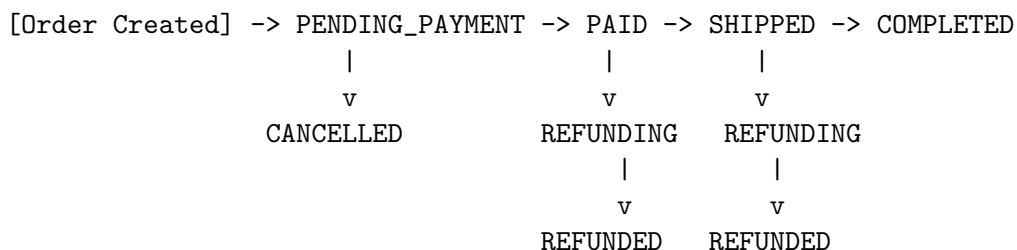
1. **Main purpose:** Before creating an order, determine whether the user's product is valid (status on sale) and whether the selected SKU has enough inventory.
2. **Conditions for cannot submit:**
 - Product status is not `ON_SALE` (e.g., offline, deleted).
 - SKU inventory is less than or equal to 0.
 - Purchase quantity exceeds SKU inventory.
3. **If `productStatus = 'ON_SALE'`, `skuStock = 3`, `buyCount = 5`:** Returns {
 `canSubmit: false, reason: 'Purchase quantity exceeds available inventory'`
}.
4. **Additional checks:**
 - User login status: if not logged in, guide to login.
 - Purchase quantity limits: `buyCount` $\geq$ 0 and less than max per order (e.g., max 10).
 - Shipping address validation: whether default address exists or address is filled.
 - Purchase limit per user: e.g., max 2 per user; check historical and current purchases.
 - User status: whether account is disabled.

2.3 Order Status Understanding

Status definitions:

- PENDING_PAYMENT - Pending Payment
- PAID - Paid
- SHIPPED - Shipped
- COMPLETED - Completed
- CANCELLED - Cancelled
- REFUNDING - Refunding
- REFUNDED - Refunded

1. Initial status after customer submits order: PENDING_PAYMENT
2. Status after payment success: PAID
3. Status after merchant ships order: SHIPPED
4. Status if customer does not pay within payment time limit: CANCELLED
5. Statuses where refund can be requested: PENDING_PAYMENT (request to close and refund), PAID (request refund only), SHIPPED (request return and refund). COMPLETED also allows after-sales typically with time limit.
6. Order status transition flow (described):



2.4 Bug / Exception Analysis

Issue: Customer sees inventory = 1 on product detail page, but when submitting order, system shows "Insufficient inventory."

Possible Causes:

1. **Data inconsistency (most common):** Product detail page shows inventory from cache (e.g., Redis) or a non-real-time read replica. Order submission backend validates against master database real inventory. If cache or replica hasn't synced latest master data, discrepancy occurs.
2. **Concurrent purchase:** User A sees stock = 1, but before their order request reaches backend, another user B completes payment, making actual inventory 0. When A's request arrives, validation fails.

3. **Inventory pre-deduction not shown:** User A sees stock = 1. User B creates an order and locks that last item (status `PENDING_PAYMENT`), but frontend detail page query does not subtract pre-deducted inventory, still shows 1.
4. **SKU confusion:** Detail page shows total product inventory (e.g., all colors combined = 1), but user selects a specific SKU (e.g., "Red") and order submission validates against that SKU's inventory (could be 0). Detail page should show selected SKU inventory.

Investigation Steps:

1. Reproduce and get key info: Ask QA for specific user ID, product ID, SKU ID (if any), and exact time of issue.
2. Log analysis: Check backend API logs for order submission around that time, find the user's request, confirm the real-time inventory (`actual_stock`) at time of validation.
3. Data verification:
 - Query master database to confirm actual inventory for that SKU at that time.
 - Query replica database or cache to confirm value returned to frontend.
 - If inconsistent: root cause is data sync delay or cache update issue. Check data sync / cache invalidation logic.
 - If consistent: stock was indeed 1, problem not data inconsistency.
4. Concurrency check: Check logs for other users successfully creating orders for the same SKU within the same second or milliseconds. If yes, concurrency issue. Need to optimize inventory deduction logic (e.g., optimistic lock, pessimistic lock, Redis atomic operations).
5. Pre-deduction logic check: Examine SQL or logic querying inventory for frontend. Check if it includes locked quantity from `PENDING_PAYMENT` orders. Correct available stock formula: `total_stock - locked_stock`.
6. Conclusion and fix:
 - If cache issue: Fix cache update logic to ensure real-time refresh after inventory changes.
 - If concurrency issue: Add distributed lock for same product/order to ensure serial processing.
 - If pre-deduction issue: Unify "available stock" calculation across all places.
7. **Outcome:** Identify root cause and implement fix.

3 Data and Metrics

3.1 Core Metrics Design

Metric	Meaning	Why It Matters
Daily Active Users (DAU)	Unique users visiting app daily	Measures overall platform scale and health.

Product Detail Page Conversion Rate	Number of users who submit orders / number of users who view product detail page	Measures attractiveness and smoothness from browsing to ordering; core conversion metric.
Add-to-Cart Rate	Users who add to cart / users who view product detail page	Measures initial product appeal; higher-funnel metric than order conversion; detects issues early.
Payment Success Rate	Paid orders / submitted orders	Directly measures success of core transaction step. High payment failure rate may indicate payment gateway issues or user flow problems.
Inventory Turnover Days	Average inventory / average daily units sold	Measures inventory management efficiency. High days indicate overstock and capital tie-up; low days risk stock-outs. Operations focus.
Average Order Value (AOV)	Total GMV / total number of orders	Measures spending per transaction. Useful for evaluating promotions like "free shipping threshold" or "bundle deals".
Stockout Rate	Number of SKUs with stock-out / number of SKUs requested	Measures inventory planning accuracy. High rate leads to poor user experience and lost sales.
Order Cancellation Rate	Orders cancelled by user / total orders	Measures firmness of purchase intention. High rate may indicate impulse buying, payment flow obstacles, or mismatch between product info and expectation.